

Ex. 2.7 — Three papers from the “Scheduling” group

Paper 2-3 — Waldspurger and Weihl, *Lottery Scheduling* (OSDI 1994)

Summary. Lottery scheduling represents resource rights as *tickets* and settles each quantum by drawing one at random: a client holding t of T tickets wins with probability $p = t/T$, so throughput tracks the allocation, no non-zero holding can starve, and a reallocation takes effect at the very next decision. The abstractions built on the draw matter more than the draw itself. *Ticket transfers* hand a blocked client’s rights to whoever is computing on its behalf, dissolving priority inversion much as priority inheritance does. *Currencies* denominate tickets locally and convert through an exchange rate into a conserved base currency, so inflation inside one trust boundary cannot leak out. *Compensation tickets* multiply a client’s funding by $1/f$ when it used only a fraction f of its quantum, so a thread that yields early for I/O is not quietly punished. A Mach 3.0 prototype (25 MHz MIPS DECStation, 100 ms quantum) reports a 2:1 Dhrystone allocation yielding 25378 against 12619 iterations/s (2.01:1), an 8:3:1 database allocation reproduced in response time as well as throughput (17.19, 43.19 and 132.20 s), and currencies insulating one user’s inflation from another (aggregate ratio 1.01:1 before, 1.00:1 after).

Critical judgment. The evaluation’s soft spot is one the paper half-concedes. Individual 60-second runs scatter badly, a 10:1 allocation being observed at 13.42:1, and the authors’ reply is that large ratios “converge toward their allocated values over longer time intervals”, offering a 20:1 allocation that averaged 19.08:1 over three minutes. That answer costs them their framing: the introduction argues interactive systems need control “at a time scale of milliseconds to seconds”, and the related-work section dismisses fair-share schedulers precisely for delivering fairness only “on a time scale of minutes”. Randomisation makes the error fall only as $1/\sqrt{n}$, their own coefficient of variation $\sqrt{(1-p)/np}$, so on the timescale the paper says matters it supplies roughly what it criticises.

The overhead claim is thinner than its conclusion. The prototype ran 2.7% slower on three Dhrystone tasks and 0.8% slower on eight, and the authors honestly note the run-to-run standard deviation was as large as the effect; the one significant measurement, 1.7% *faster* on the database server, is attributed in a footnote to incidental cache and TLB locality. “Efficient lottery scheduling does not pose any challenging problems” therefore rests on data that mostly cannot separate the two schedulers. Currencies, finally, are what make inflation safe, yet in the measured artifact they are unprotected: the user commands are setuid root, and the access-control lists deciding who may inflate a currency are proposed but left as what “a complete lottery scheduling system should” provide.

Paper 2-4 — Waldspurger and Weihl, *Stride Scheduling* (MIT/LCS/TM-528, 1995)

Summary. Stride scheduling is the same authors’ deterministic replacement for their own dice. Each client holds a *stride* inversely proportional to its tickets and a *pass* value; the scheduler always runs the minimum-pass client and advances that client’s pass by its stride, in $O(\lg n_c)$ time with a priority queue. A partial quantum advances the pass by only $f \cdot \text{stride}$, while joins, leaves and ticket changes are handled by saving and rescaling the client’s unconsumed remainder against a smoothly advancing global pass. Because each client’s state is independent, the transfers, inflation and currencies of paper 2-3 all survive on the deterministic substrate. The error bounds are the argument: lottery scheduling’s absolute error grows as $O(\sqrt{n_a})$ in the number of allocations,

whereas stride’s pairwise relative error never exceeds one quantum whatever n_a is. Absolute error can still reach $O(n_c)$ under skew, 101 clients at 100:1:…:1 giving an error of 50, which motivates *hierarchical* stride scheduling: a balanced tree of aggregated groups bounding absolute error to $O(\lg n_c)$, which measured 4.5 on that example. In simulation at 19:1 over one million allocations, the one-ticket client saw response times of $\mu = 20.00$, $\sigma = 0.01$ quanta under stride against $\mu = 20.13$, $\sigma = 19.64$ and a range of 1–194 quanta under lottery. Linux 1.1.50 prototypes held CPU ratios within 1% of ideal (3:1 gave 3.001:1) and UDP ratios within 5%, in under 300 lines each.

Critical judgment. The head-to-head case against lottery scheduling is made entirely in simulation. The prototypes are measured against the *ideal* ratio and against stock Linux, never against a lottery-scheduled kernel, so the headline claim of significantly improved accuracy over lottery is nowhere demonstrated on hardware. They also leave the modularity argument unexercised: Section 3 shows how transfers and currencies would be built, but the CPU prototype implements neither, so the abstractions that justify the design are never run. The overhead figure is softer than it reads, because the authors concede their prototype does a linear scan rather than the $O(\lg n_c)$ structure the complexity analysis assumes; the cheap measurement and the good bound describe different programs.

Against that, the paper is unusually candid and criticism should not pretend otherwise. It reports that the network result required lowering a hard-coded delay constant in `ttcp`, without which ratios were “consistently lower than specified”; it volunteers that its core algorithm turned out to be nearly VirtualClock and WFQ rediscovered; and its conclusion concedes that lottery scheduling is “conceptually simpler” and “effectively stateless” while stride demands careful state fixups on every dynamic change. History settled the trade: Linux’s CFS and its EEVDF successor both schedule by picking the minimum-virtual-time task and advancing it by a weight-scaled increment, stride’s rule under another name, while no production kernel ever shipped the lottery.

Paper 2-1 — Buttazzo, *Rate Monotonic vs. EDF: Judgment Day* (Real-Time Systems 29(1), 2005)

Summary. This is a rebuttal rather than a new result. Buttazzo takes the five claims habitually used to prefer Rate Monotonic to Earliest Deadline First (simpler implementation, lower overhead, easier analysis, more predictable overload, less jitter) and answers each with theory, counterexamples and simulation. He grants RM one real point: on a commercial kernel with at most 256 priority levels, a new deadline falling between two adjacent levels can force every active deadline to be remapped, and only RM admits $O(1)$ insertion via one FIFO per level. Written natively, the two differ only in the ready-queue sort key. On overhead he argues the accounting is backwards: EDF pays one deadline computation per job, but RM forces far more preemptions, and his simulations show preemption counts rising roughly linearly with load under RM while *falling* under EDF at high load, because a long job holding the earliest deadline can push a successor past an arrival and avoid a preemption outright. On analysis, EDF is exact in $O(n)$ at $U \leq 1$ when deadlines equal periods, while RM’s $\ln 2 \approx 0.69$ bound needs pseudo-polynomial response-time analysis to become exact. Under permanent overload EDF rescales every period to $T_i U$ while RM can block low-priority tasks outright; under transient overload a 1.5-unit overrun in the *highest*-rate task makes τ_2 miss rather than the longest-period τ_4 , refuting the belief that RM fails predictably from the bottom up. A three-task example gives jitters of 0, 2, 8 under RM against 1, 2, 3 under EDF.

Critical judgment. The method is asymmetric in a way the paper notices locally and forgets globally. A counterexample refutes “RM always wins”; it cannot establish “EDF wins”, and But-

tazzo says exactly that of his jitter example, which “does not prove that EDF always introduces less jitter”, yet the abstract and conclusions generalise anyway. The overhead argument is the weakest link: Section 3 counts preemptions, never measures a context switch on real hardware, and never nets the saved switches against EDF’s per-job deadline update, so the verdict rests on a proxy measured over synthetic task sets. Two things that would decide the industrial argument are absent: multiprocessors, and the certification and tooling ecosystem built on fixed priorities, which he reduces to a circularity.

He does not, however, stack the deck. He grants RM the $O(1)$ ready queue and the cheaper exact test when deadlines are shorter than periods, grants that RM confines an overrun to tasks of lower priority while EDF confines it nowhere, and concludes both that which overload behaviour is preferable is “highly application dependent” and that the two algorithms are “not very well suited” to overload at all, the remedy being resource reservation. That concession aged best: Linux’s `SCHED_DEADLINE` (merged in 3.14, 2014) ships EDF together with the Constant Bandwidth Server, the Abeni–Buttazzo mechanism of his own Section 7.3, rather than EDF alone.

All three argue the same way: 2-4 unseats its own predecessor’s randomness, and 2-1 unseats static rate priorities, each on the ground that a scheduler should carry the extra state needed to decide accurately.

References

- [1] C. A. Waldspurger and W. E. Weihl. “Lottery Scheduling: Flexible Proportional-Share Resource Management.” *Proceedings of the First USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Monterey, CA, November 1994. (Paper list ID 2-3.)
- [2] C. A. Waldspurger and W. E. Weihl. “Stride Scheduling: Deterministic Proportional-Share Resource Management.” Technical Memorandum MIT/LCS/TM-528, MIT Laboratory for Computer Science, June 1995. (Paper list ID 2-4.)
- [3] G. C. Buttazzo. “Rate Monotonic vs. EDF: Judgment Day.” *Real-Time Systems*, 29(1):5–26, 2005. (Paper list ID 2-1.)